STRUCTURAL DESIGN PATTERN

Bridge Design Pattern in C#

Decoupling Abstraction from Implementation: Solving Cartesian Class Explosions with Object Composition.

1. The Problem: The Remote Control & Device Dilemma

The **Bridge Pattern** solves the classic class explosion that occurs when a hierarchy grows in two independent directions at the same time: **Controls** (what the user interacts with) and **Devices** (the hardware that executes commands).

> **⚠ Anti-Pattern: Combining Two Independent Hierarchies with Inheritance**
>
> Imagine you are building a universal controller app. You have two remotes: **BasicRemote** (power on/off) and **AdvancedRemote** (adds a one-touch mute button). You also have two appliances: **TV** and **Radio**.
> If you use inheritance, you must create a class for every combination: `BasicRemoteTv`, `BasicRemoteRadio`, `AdvancedRemoteTv`, and `AdvancedRemoteRadio`. Adding a **Projector** forces you to write 2 new remote classes. Adding a **SmartRemote** forces you to write 3 more classes for every device!

❌ Before: The M × N Inheritance Explosion

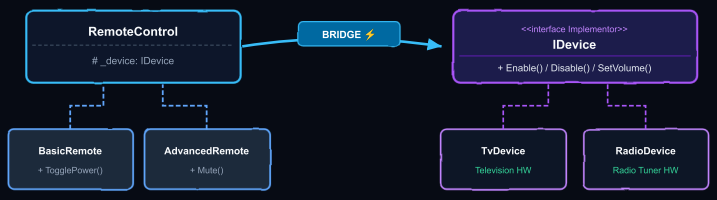
```
// Rigid inheritance: Every combination demands a brand new subclass
public abstract class RemoteControl { public abstract void TogglePower(); }

public class BasicRemoteTv : RemoteControl { public override void TogglePower() { /* TV logic */ } }
public class BasicRemoteRadio : RemoteControl { public override void TogglePower() { /* Radio logic */ } }
public class AdvancedRemoteTv : RemoteControl { public override void TogglePower() { /* TV logic */ } }
public class AdvancedRemoteRadio : RemoteControl { public override void TogglePower() { /* Radio logic */ } }
// Adding Projector requires 2 classes. Adding SmartRemote requires 3 more classes!
```

> **✅ The Bridge Solution: Decouple Remotes from Devices**
>
> Separate them into two distinct hierarchies: **Remotes** (Abstraction) and **Devices** (Implementor). The base remote holds a reference to an `IDevice` interface. That reference is the **Bridge**. Now, classes grow at **$M + N$** instead of $M \times N$.

2. Architectural Structure (Excalidraw-Style Diagram)

The structural relationship showing the Remote hierarchy (Abstraction) connected to the Device hierarchy (Implementor) through a single composition reference:



3. Structural Component Breakdown

Component	In our Example	Architectural Responsibility
Abstraction	`RemoteControl`	High-level interface used by the client. Holds the bridge reference (`IDevice`).
Refined Abstraction	`BasicRemote`, `AdvancedRemote`	Adds user-facing features (e.g. `Mute()`) by orchestrating device primitives.
Implementor	`IDevice`	The hardware contract declaring low-level primitives: power states and volume dials.
Concrete Implementors	`TvDevice`, `RadioDevice`	Specific physical hardware drivers that carry out the actual operations.

> 💡 **Why this is called the Bridge**
>
> The bridge is the reference field `protected readonly IDevice _device` inside the Remote. You can hand an `AdvancedRemote` to a TV or a Radio at runtime without touching either class.

4. Clean & Simple C# Implementation

A complete, runnable example showing how remotes and devices operate independently via the bridge reference:

✅ Clean C#: RemoteControl (Abstraction) + IDevice (Implementor)

```
namespace BridgePatternSimple;

// 1. Implementor Hierarchy (The Hardware Device Contract)
public interface IDevice {
    bool IsEnabled { get; }
    void Enable();
    void Disable();
    void SetVolume(int percent);
}

public class TvDevice : IDevice {
    public bool IsEnabled { get; private set; }
    public void Enable()  { IsEnabled = true;  Console.WriteLine("[TV] Screen powered ON (4K Display active)");
}
    public void Disable() { IsEnabled = false; Console.WriteLine("[TV] Screen powered OFF (Standby mode)"); }
    public void SetVolume(int pct) => Console.WriteLine($"[TV] Speakers set to volume: {pct}%");
}

public class RadioDevice : IDevice {
    public bool IsEnabled { get; private set; }
    public void Enable()  { IsEnabled = true;  Console.WriteLine("[Radio] Tuner ON (FM 101.5 MHz playing)"); }
    public void Disable() { IsEnabled = false; Console.WriteLine("[Radio] Tuner powered OFF"); }
    public void SetVolume(int pct) => Console.WriteLine($"[Radio] Audio dial set to: {pct}%");
}

// 2. Abstraction Hierarchy (The Remote Controls)
public class BasicRemote {
    protected readonly IDevice _device; // <--- THE BRIDGE TO THE HARDWARE

    public BasicRemote(IDevice device) => _device = device;

    public void TogglePower() {
        if (_device.IsEnabled) _device.Disable();
        else _device.Enable();
    }

    public void VolumeUp() => _device.SetVolume(50);
}

public class AdvancedRemote : BasicRemote {
    public AdvancedRemote(IDevice device) : base(device) { }

    public void Mute() {
        Console.WriteLine("[Remote] One-touch Mute button pressed:");
        _device.SetVolume(0);
    }
}

// 3. Execution (Mixing & Matching Any Remote with Any Device)
public static class Program {
    public static void Main() {
        // Advanced remote controlling a TV:
        IDevice tv = new TvDevice();
        var remote = new AdvancedRemote(tv);
        remote.TogglePower(); // Turns TV on
        remote.Mute();        // Mutes TV

        // Swapping to a Radio using the exact same remote class:
        IDevice radio = new RadioDevice();
        var radioRemote = new AdvancedRemote(radio);
        radioRemote.TogglePower(); // Turns Radio on
        radioRemote.Mute();        // Mutes Radio
    }
}
```

5. Real-World Use Cases in ASP.NET Core & .NET

🛠 Use Case 1: ASP.NET Core Logging Infrastructure (ILogger & Sinks)

`ILogger<T>` represents the **Abstraction** (developers call high-level methods like `LogInformation()`, `BeginScope()`). `ILoggerProvider` and sink handlers (Console, Application Insights, Seq, Serilog) serve as the **Implementors**. Application developers evolve logging calls without caring what target storage formats consume them.

🛠 Use Case 2: Multi-Platform Storage & Cloud File Providers

An enterprise CMS exposes a `MediaAssetService` (Abstraction with methods like `PublishImage()`, `ArchiveVideo()`). It bridges to an `IBlobStorageDriver` (Implementor with implementations for **Azure Blob Storage**, **AWS S3**, and **LocalDiskStorage**). Swapping or adding cloud storage providers never breaks business publishing logic.

6. Trade-offs & Architectural Comparison Matrix

✓ Pros (Advantages)

- **Prevents Class Bloat:** Scales at $M + N$ instead of explosive $M \times N$ class proliferation.
- **Single Responsibility:** High-level logic focuses on domain intent; Implementor focuses on low-level drivers.
- **Runtime Swapping:** Implementors can be injected and switched at runtime via Dependency Injection.

✗ Cons (Trade-offs)

- **Initial Indirection:** Increases upfront code complexity for small domains that will never have multiple platforms.
- **Cohesive Interface Design:** Requires designing a clean, platform-neutral Implementor contract early on.

💡 Bridge vs. Adapter vs. Strategy

• **Bridge:** Designed *upfront* to let two orthogonal dimensions (Abstraction & Implementor) evolve independently.
• **Adapter:** Applied *retrospectively* to make two incompatible, pre-existing interfaces work together.
• **Strategy:** Swaps specific *algorithms/behaviors* (how an operation is computed), whereas Bridge decouples an entire class hierarchy from low-level execution platforms.